

Stack Problems and Solutions

1. Valid Parentheses

Problem:

Given a string of parentheses, check if it is valid (i.e., open parentheses must be closed in the correct order).

Approach 1: Stack-based solution

- Trick to Identify:

- The problem deals with matching pairs and ordering. A stack is the best choice when order matters in pairing operations.

Steps:

- Use a stack to track open parentheses.
- Push opening brackets ([, {, [) onto the stack.
- For closing brackets, check if the stack's top matches the corresponding opening bracket.
- If they don't match or the stack is empty, return false.
- Time Complexity: $O(n)$, where n is the length of the string.
- Space Complexity: $O(n)$, for the stack.

Pseudocode:

```
bool isValid(string s) {  
    stack st;  
    for (char c : s)
```

```

{
  if (c == '(' or c == '{' or c == '[') {
    st.push(c);
  } else {
    if (st.empty()) return false;
    if ((c == ')' && st.top() != '(') or
        (c == '}' && st.top() != '{') or
        (c == ']' && st.top() != '[')) return false;
    st.pop();
  }
}
return st.empty();
}

```

Dry Run:

Input: $s = \{ [()] \}$

Steps:

$st = []$ (initially empty).

$\{ \rightarrow \text{push} \rightarrow st = [\{]$

$[\rightarrow \text{push} \rightarrow st = [\{, '[$

$(\rightarrow \text{push} \rightarrow st = [\{, '[, '(']$

$) \rightarrow \text{pop} \rightarrow st = [\{, '[$

$] \rightarrow \text{pop} \rightarrow st = [\{]$

$\} \rightarrow \text{pop} \rightarrow st = []$

□

Output: true (stack is empty).

2. Min Stack

Problem:

Design a stack that supports push, pop, top, and retrieving the minimum element in constant time.

Approach 1: Two Stacks

- Trick to Identify:
- Use an additional stack to keep track of the minimum elements at each stage.

Steps:

- Maintain two stacks: one for the elements and another for the minimums.
- Push operations will also update the min stack.
- Pop operations will remove elements from both stacks.
- Time Complexity: $O(1)$ for all operations.
- Space Complexity: $O(n)$, where n is the number of elements in the stack.

Pseudocode:

```
class MinStack {  
    stack st,
```

```
minSt;
```

```
public:
```

```
void push(int val) {
```

```
    st.push(val);
```

```
    if (minSt.empty() or val <= minSt.top()) minSt.push(val);  
}
```

```
void pop() {
```

```
    if (st.top() == minSt.top()) minSt.pop();
```

```
    st.pop();
```

```
}
```

```
int top() {
```

```
    return st.top();
```

```
}
```

```
int getMin() {
```

```
    return minSt.top();
```

```
}
```

```
};
```

Dry Run:

Operations:

push(5) → st = [5], minSt = [5]

push(3) → st = [5, 3], minSt = [5,

3]

`getMin()` $\rightarrow$ `minSt.top()` = 3

`pop()` $\rightarrow$ `st = [5]`, `minSt = [5]`

Output: 3

3. Evaluate Reverse Polish Notation

Problem:

Evaluate the value of a mathematical expression in reverse Polish notation (RPN).

Approach 1: Stack-based solution

- Trick to Identify:
- RPN requires processing operators after their operands, making stacks ideal for evaluation.

Steps:

- Traverse the tokens in the input.
- Push numbers onto the stack.
- When encountering an operator, pop two numbers, evaluate the result, and push it back onto the stack.
- Time Complexity: $O(n)$, where n is the number of tokens.
- Space Complexity: $O(n)$, for the stack.

Pseudocode:

```
int evalRPN(vector& tokens)
```

```

{
stack st;
for (string& token : tokens) {
if (isdigit(token.back())) {
st.push(stoi(token));
} else {
int b = st.top(); st.pop();
int a = st.top(); st.pop();
if (token == "+") st.push(a + b);
else if (token == "-") st.push(a - b);
else if (token == "*") st.push(a * b);
else if (token == "/") st.push(a / b);
}
}
return st.top();
}

```

Dry Run:

Input: tokens = ["2", "1", "+", "3", "*"]

Steps:

st = [] (initially empty).

2 → push → st = [2]

1 → push → st = [2, 1]

+ → pop 1, 2, evaluate $2 + 1 = 3$ → push → st =

[3]

3 → push → st = [3, 3]

* → pop 3, 3, evaluate $3 * 3 = 9$ → push → st = [9]

Output: 9

4. Generate Parentheses

Problem:

Given n , generate all combinations of n pairs of valid parentheses.

Approach 1: Backtracking

- Trick to Identify:

When the problem involves generating all combinations, backtracking is often the best approach.

Steps:

- Use recursion to generate all valid combinations.
- Track the number of open and close parentheses.
- Ensure that at any point, the number of open parentheses is at least as many as close ones.

- Time Complexity: $O(4^n / \sqrt{n})$: Catalan number complexity.

- Space Complexity: $O(n)$: For recursion

stack.

Pseudocode:

```
void generate(int open, int close, string current, vector& result) {  
    if (open == 0 && close == 0) {  
        result.push_back(current);  
        return;  
    }  
    if (open > 0) generate(open - 1, close, current + "(", result);  
    if (close > open) generate(open, close - 1, current + ")", result);  
}
```

```
vector generateParenthesis(int n) {  
    vector result;  
    generate(n, n, "", result);  
    return result;  
}
```

Dry Run:

Input: $n = 2$

Steps:

Start: $open=2, close=2, current=""$

Add ($\rightarrow open=1, close=2, current="("$

Add ($\rightarrow open=0, close=2,$

current="("

Add $\rightarrow$ open=0, close=1, current="()"

Add $\rightarrow$ open=0, close=0, current="()" $\rightarrow$ Result: ["()"]

Backtrack, add $\rightarrow$ earlier: open=1, close=1, current="()" $\rightarrow$ Complete: ["()", "()"]

Output: ["()", "()"]

5. Daily Temperatures

Problem:

Given a list of temperatures, find how many days you have to wait for a warmer temperature.

Approach 1: Brute Force

Steps:

- For each day, check subsequent days for a warmer temperature.
- Count the days or return 0 if none exists.
- Time Complexity: $O(n^2)$.
- Space Complexity: $O(1)$.

Approach 2: Monotonic Stack

- Trick to Identify:
- For problems with comparisons between previous and next elements, a monotonic stack is

optimal.

Steps:

- Traverse the array from the end to the start.
- Use a stack to store indices of temperatures in decreasing order.
- Update results based on the stack.
- Time Complexity: $O(n)$.
- Space Complexity: $O(n)$.

Pseudocode:

```
vector dailyTemperatures(vector& temperatures) {  
    int n = temperatures.size();  
    vector result(n, 0);  
    stack st;  
  
    for (int i = n - 1; i >= 0; i--) {  
        while (!st.empty() && temperatures[st.top()] <= temperatures[i]) st.pop();  
        result[i] = st.empty() ? 0 : st.top() - i;  
        st.push(i);  
    }  
    return result;  
}
```

Dry Run:

Input: temperatures = [73, 74, 75, 71, 69, 72, 76,

73]

Steps:

$i = 7$: Stack is empty $\rightarrow$ result[7] = 0

$i = 6$: Stack $\rightarrow$ [7] $\rightarrow$ result[6] = 0

$i = 5$: Stack $\rightarrow$ [6, 7] $\rightarrow$ result[5] = 1

Continue updating...

Output: [1, 1, 4, 2, 1, 1, 0, 0]

6. Car Fleet

Problem:

Calculate the number of car fleets reaching the destination given their speeds and starting positions.

Approach 1: Sorting and Simulation

- Trick to Identify:

- When calculating "events," sort the elements to evaluate conditions in order.

Steps:

- Pair positions and speeds, and sort by position in descending order.

- Simulate each car's time to reach the destination.

- Count the number of fleets based on time.

- Time Complexity: $O(n \log n)$, for sorting.

- Space Complexity:

$O(n)$.

Pseudocode:

```
int carFleet(int target, vector& position, vector& speed) {  
    int n = position.size();  
    vector< car> cars;  
    for (int i = 0; i < n; i++) cars.push_back({position[i], speed[i]});  
    sort(cars.rbegin(), cars.rend());
```

```
    int fleets = 0;  
    double time = 0;  
    for (auto& car : cars) {  
        double current_time = (double)(target - car.first) / car.second;  
        if (current_time > time) {  
            time = current_time;  
            fleets++;  
        }  
    }  
    return fleets;  
}
```

Dry Run:

Input: target = 12, position = [10, 8, 0, 5, 3], speed = [2, 4, 1, 1,

3]

Steps:

Sort: [(10, 2), (8, 4), (5, 1), (3, 3), (0, 1)]

Calculate times: [1.0, 1.0, 7.0, 3.0, 12.0]

Fleets: 3

Output: 3

7. Largest Rectangle in Histogram

Problem:

Find the largest rectangle area in a histogram represented by an array of heights.

Approach 1: Brute Force

Steps:

- Iterate over all pairs of bars.
- Calculate the area for each range and track the maximum.
- Time Complexity: $O(n^2)$.
- Space Complexity: $O(1)$.

Approach 2: Stack

- Trick to Identify:
- Problems involving "nearest smaller element" or "largest area" often use

stacks.

Steps:

- Use a stack to maintain increasing heights.
- Pop from the stack to calculate areas when the current height is smaller.
- Ensure left and right bounds are correctly set for each bar.
- Time Complexity: $O(n)$.
- Space Complexity: $O(n)$.

Pseudocode:

```
int largestRectangleArea(vector& heights) {  
    stack st;  
    int max_area = 0, n = heights.size();  
  
    for (int i = 0; i <= n; i++) {  
        while (!st.empty() && (i == n or heights[st.top()] > heights[i])) {  
            int h = heights[st.top()];  
            st.pop();  
            int width = st.empty() ? i - st.top() - 1;  
            max_area = max(max_area, h * width);  
        }  
        st.push(i);  
    }  
    return
```